

Laboratory 2:

Signals and Systems

Signal Transformations: Time Shift, Amplitude Scaling, and Time Reversal

Hardware Implementation with Arduino and MATLAB Verification

Lab Duration: 2 hours

Pre-Lab Time Expected: 40 minutes (mathematical derivations and parameter assignment)

Key Concept: Implementing fundamental signal domain and range transformations in embedded hardware and validating their fidelity against ground-truth MATLAB operators.

Contents

1	Core Concepts: Continuous and Discrete Signal Transformations	7
1.1	Fundamental Continuous-Time Signal Transformations	7
1.1.1	Time Shift (Delay and Advance)	7
1.1.2	Amplitude Scaling	7
1.1.3	Time Reversal	7
1.1.4	Combined Transformations and Order of Operations	8
1.2	Discrete-Time Analogues and MATLAB Operators	8
1.3	Exponential and Sinusoidal Signals	8
1.3.1	Real Exponential Signals	8
1.3.2	Sinusoidal Signals	8
1.3.3	Damped Sinusoidal Signals	9
1.4	Unit Impulse and Unit Step Functions	9
1.4.1	Unit Impulse (Dirac Delta)	9
1.4.2	Unit Step Function	9
2	Pre-Lab Assignment	10
3	Equipment & Setup	11
3.1	Potentiometer Wiring and Gain Calculation	11
4	Laboratory Procedure	13
4.1	Submission Evidence Map	13
4.2	Phase 1: Base Signal Generation and Dual-Channel Acquisition	13
4.2.1	Step 1.1: Load the TA-Assigned Base Signal	13
4.2.2	Step 1.2: Acquire the Base Signal into MATLAB	15
4.3	Phase 2: Hardware Implementation of Signal Transformations	17
4.3.1	Step 2.1: Amplitude Scaling via Potentiometer	17
4.3.2	Step 2.2: Time Shift via Circular Sample Buffer	18
4.3.3	Step 2.3: Time Reversal via Reversed LUT Indexing	20
4.3.4	Step 2.4: Combined Transformation — Reversal, Shift, and Scale	21
4.4	Phase 3: MATLAB Verification and Predicted vs. Measured Overlay	23

4.4.1	Step 3.1: Acquire All Transformed Signals into MATLAB	23
4.4.2	Step 3.2: Predicted vs. Measured Overlay Plots	23
4.4.3	Step 3.3: Residual Error Quantification	24
5	Data Analysis	26
5.1	Analysis Step 1: Pre-Lab Sketch vs. Hardware vs. MATLAB Comparison	26
5.2	Analysis Step 2: Signal Symmetry Classification	26
5.3	Analysis Step 3: <code>circshift</code> Boundary Artifact Investigation	26
5.4	Analysis Step 4: Unit Step Windowing Verification	26
5.5	Analysis Step 5: Error Source Ranking	26
6	Grading Rubric	28
	References and Inspiration	29

About This Lab Manual

Note on Media Placeholders: This lab manual contains boxed placeholders labeled “[INSERT ...]” that indicate where professional reference media should be embedded by instructors. These are **NOT** spaces for students to submit content. Examples include:

- **Reference circuit schematics** - showing correct hardware wiring configurations
- **Example oscilloscope screenshots** - demonstrating expected waveforms and measurements
- **Example MATLAB plots** - showing typical analysis outputs
- **Theoretical diagrams and block diagrams** - pedagogical reference material

Student-submitted materials (circuit photographs, measured data, MATLAB code outputs) are explicitly requested in **Deliverable** and **Checkpoint** sections and must include your name and student ID.

Grading and Authenticity Requirements

Deliverable (1/12): This is a graded laboratory assignment. Include your **name and student ID** in all required screenshots and photographs (oscilloscope captures, MATLAB plots, bread-board/setup photos).

Deliverable (2/12): All signal transformations and measurements must be acquired from your own hardware build and unique TA-assigned parameters. Pre-computed or borrowed data is not accepted.

Checkpoint (1/8): Before submission, verify that each MATLAB figure includes axis labels with units, a grid, a title with your name/ID, and a legend (if applicable). Oscilloscope photos must show clear identification of input vs. output signals.

Title & Real-World Context

You are a Junior Signal Processing Engineer at a Major Seismic Data Acquisition Firm

Your technical lead has assigned you a critical role in validating the signal processing pipeline for a next-generation **marine seismic acquisition system** deployed in deepwater oil and gas exploration. The system must reliably handle three demanding operational scenarios:

1. **Seismic Reflection Survey Processing (Primary Application).** Your company deploys towed streamer arrays (up to 12 km long) carrying thousands of hydrophone receivers to image subsurface petroleum reservoirs. Each receiver channel must compensate for **time delays** caused by cable length, water depth, and sensor positioning, then apply **amplitude corrections** for water absorption and geometrical spreading losses. A single miscalibrated time delay (off by just microseconds) or incorrect amplitude scaling can destroy the lateral coherence of the seismic image and cause geophysicists to miss fault structures worth millions in recoverable reserves. Your signal transformation pipeline must achieve $<1\%$ amplitude error and <10 microsecond timing precision across all 5,000 channels during a 6-hour acquisition.
2. **Wavefield Reconstruction and Time-Reversal Focusing (Advanced Processing).** Post-acquisition, your processing center uses **time-reversed** recordings to reconstruct seismic wavefields and focus energy back to the subsurface source. This technique (time-reversal focusing) improves lateral resolution by effectively creating a synthetic “wide-aperture” receiver. Errors in time-reversal implementation or amplitude normalization corrupt the reconstructed field and degrade final image quality. Your hardware validation ensures that production processing nodes compute reversals accurately and with minimal latency (≤ 2 ms per 1000-sample trace).
3. **Real-Time Multi-Channel Gain Normalization (Operational Constraint).** As raw data streams from the cable, a Field Control Unit (FCU) must apply **real-time amplitude scaling** to each channel to compress an 8-decade dynamic range down to the 12-bit ADC range before storage. Time-varying gain adjustments (following an exponential decay profile due to water absorption) must be synchronized across all channels. Any drift or quantization in the scaling operation causes amplitude spikes, dead channels, or energy loss that contaminates the final product and requires costly reacquisition.

Your Mission. In this lab, you will validate the three fundamental signal transformations on a real-time embedded platform and demonstrate their correctness against production-grade MATLAB processing routines:

- Implement **time shift** via circular buffering (emulating cable-delay compensation).
- Implement **amplitude scaling** via potentiometer control (emulating gain normalization).
- Implement **time reversal** via LUT index inversion (emulating field reconstruction).
- Cross-check hardware outputs against MATLAB operators (`circshift`, scalar multiply, `flip1r`) used in production processing.
- Quantify residual errors and propose mitigation strategies for deployment.

The petroleum industry runs on the assumption that field-acquired seismic data is mathematically correct and traceable. A single systematic error in signal transformation—introduced silently in

an embedded device—can propagate through months of processing and surface only when geophysicists discover imaging artifacts that are nearly impossible to debug. Your rigorous validation approach (hardware + MATLAB correlation, error quantification, edge-case testing) is the professional standard for certifying mission-critical seismic processing equipment.

Constraints (Fixed). 2-hour lab execution window, existing hardware platform (Arduino Uno, oscilloscope, MATLAB). Unique TA-assigned base signal parameters ensure independent work. All skeleton code provided to focus on transformation logic rather than low-level firmware details. All checkpoints and deliverables remain exactly as specified herein.

Upon completion of this laboratory, you will demonstrate:

1. **Mathematical Transformation Formulation.** Derive, in closed form, the time-shifted $x(t - t_0)$, amplitude-scaled $B \cdot x(t)$, and time-reversed $x(-t)$ versions of a given base signal, and map those expressions to discrete-time algorithms suitable for embedded implementation.
2. **Hardware Transformation Implementation Under Resource Constraints.** Code and deploy time-shift via circular-buffer indexing, amplitude scaling via analog potentiometer input, and time-reversal via reversed LUT lookup, while meeting deterministic timing and memory requirements of a real-time embedded system.
3. **Dual-Domain Validation and Error Quantification.** Simultaneously acquire transformed signals via oscilloscope (analog ground truth) and MATLAB serial acquisition, overlay measurements against theoretical predictions, and quantify residual errors by transformation type (RMS error, peak error, boundary artifacts).
4. **Discrete-Time Operator Equivalence and Correctness.** Establish the analytical and empirical correspondence between hardware operations and MATLAB standard operators (`circshift`, scalar multiply, `fliplr`), and explain why each discrete operator is the correct computational analogue of the continuous transformation.
5. **Signal Family Parameterization and Transformation Effects.** Identify and measure the defining parameters (A , α , ω_0 , ϕ) of real exponentials, sinusoids, and damped sinusoids from hardware-acquired data, and predict how each transformation affects those parameters mathematically and empirically.
6. **Production-Grade Error Analysis and Mitigation.** Identify systematic error sources (quantization, boundary wrap-around, potentiometer nonlinearity, jitter), rank them by magnitude, and propose practical mitigation strategies suitable for production seismic-processing equipment.

1 Core Concepts: Continuous and Discrete Signal Transformations

Signal transformations are the fundamental algebraic operations by which the domain (time axis) and range (amplitude axis) of a signal are manipulated. Mastery of these operations is prerequisite to every subsequent topic in this course—convolution, system responses, Fourier analysis, and filtering—because each of those computational procedures can be decomposed into and understood through elementary transformations. In production signal processing systems (seismic, radar, communications), systematic errors in transformation implementation frequently escape detection until they compound across millions of samples and render the final output useless.

1.1 Fundamental Continuous-Time Signal Transformations

Let $x(t)$ denote an arbitrary continuous-time base signal. The three elementary independent-variable and dependent-variable transformations are:

1.1.1 Time Shift (Delay and Advance)

A time shift by t_0 seconds maps $x(t) \rightarrow y(t) = x(t - t_0)$. The signal retains its shape but is translated along the time axis. A positive t_0 produces a delay (the feature that was at $t = 0$ now appears at $t = t_0 > 0$); a negative t_0 produces an advance:

$$y(t) = x(t - t_0), \quad t_0 \in \mathbb{R}. \quad (1)$$

For a sinusoidal base signal $x(t) = A \cos(\omega_0 t)$, a time shift by t_0 introduces a phase offset $\phi_0 = \omega_0 t_0$:

$$y(t) = A \cos(\omega_0(t - t_0)) = A \cos(\omega_0 t - \phi_0). \quad (2)$$

Equation (2) is fundamental: it demonstrates that time delay and phase shift are equivalent representations for sinusoidal signals, a relationship that underpins Fourier-domain analysis later in the course.

1.1.2 Amplitude Scaling

Amplitude scaling multiplies the signal's range by a scalar $B \in \mathbb{R}$:

$$y(t) = B \cdot x(t). \quad (3)$$

When $|B| > 1$ the signal is amplified; when $0 < |B| < 1$ it is attenuated; when $B < 0$ the signal is simultaneously scaled and inverted (reflected about the horizontal axis). In this laboratory a potentiometer physically implements this operation by acting as a programmable voltage divider whose wiper position sets:

$$B_{\text{hw}} = \frac{V_{\text{wiper}}}{V_{\text{supply}}} = \frac{\text{analogRead(A1)}}{1023} \in [0, 1]. \quad (4)$$

1.1.3 Time Reversal

Time reversal reflects the signal about the vertical axis by replacing t with $-t$:

$$y(t) = x(-t). \quad (5)$$

For a causal signal (one that is identically zero for $t < 0$), time reversal produces a signal that is nonzero only for $t \leq 0$ —a non-causal output. Exact continuous-time reversal is therefore physically non-realizable in real time; however, it is exactly implementable on a finite-duration buffered waveform stored in memory, which is precisely how the Arduino LUT approach realizes it.

1.1.4 Combined Transformations and Order of Operations

Transformations can be composed. The general affine time transformation is:

$$y(t) = B \cdot x(at - t_0), \quad (6)$$

where B is the amplitude scale, a is the time scale factor (compression if $|a| > 1$, expansion if $|a| < 1$, reversal if $a = -1$), and t_0 is the time shift applied after time scaling. **Order of operations matters:** time scaling and shifting do not commute unless the argument is rewritten carefully. In this laboratory $a = \pm 1$ only (no compression or expansion), so the combined form reduces to:

$$y(t) = B \cdot x(\pm t - t_0). \quad (7)$$

1.2 Discrete-Time Analogues and MATLAB Operators

When the base signal is stored as a finite-length vector $\mathbf{x} = [x[0], x[1], \dots, x[N-1]]$ of N samples at sampling period T_s , the continuous-time transformations map to exact array operations:

Table 1. Continuous-Time Transformations and Their Discrete-Time MATLAB Counterparts

Transformation	CT Expression	MATLAB Discrete Equivalent
Time shift (k samples)	$x(t - kT_s)$	<code>circshift(x, k)</code>
Amplitude scale by B	$B \cdot x(t)$	<code>B * x</code>
Time reversal	$x(-t)$	<code>fliplr(x)</code>
Shift then scale	$B \cdot x(t - kT_s)$	<code>B * circshift(x, k)</code>
Reversal then shift	$x(-t - t_0)$	<code>circshift(fliplr(x), k)</code>
Combined	$B \cdot x(-t - t_0)$	<code>B * circshift(fliplr(x), k)</code>

Important: `circshift` implements a circular shift—samples that shift off one end of the array reappear at the other end. This is exact only when the signal is periodic with period NT_s . For non-periodic or transient signals, a boundary wrap-around artifact will appear; documenting, explaining, and proposing a mitigation for this artifact is part of the post-lab analysis.

1.3 Exponential and Sinusoidal Signals

The two canonical signal families used as base signals in this laboratory are:

1.3.1 Real Exponential Signals

$$x(t) = Ae^{\alpha t}, \quad A, \alpha \in \mathbb{R}. \quad (8)$$

The sign of α determines the qualitative behavior: $\alpha < 0$ gives a decaying (stable) exponential; $\alpha > 0$ gives a growing (unstable) one. The time constant $\tau = 1/|\alpha|$ is the time required for the amplitude to fall to $e^{-1} \approx 36.8\%$ of its initial value (for a decaying exponential) or to grow to $e^1 \approx 271.8\%$ (for a growing one). In hardware, α is set by the LUT pre-computation parameters assigned by your TA.

1.3.2 Sinusoidal Signals

$$x(t) = A \cos(\omega_0 t + \phi), \quad \omega_0 = 2\pi f_0. \quad (9)$$

The three defining parameters are: amplitude A (V), angular frequency ω_0 (rad/s), and initial phase ϕ (rad). Note that a pure sinusoid is neither causal nor finite-energy—it is an idealized

infinite-duration signal. In hardware it is always windowed to a finite duration equal to one LUT period.

1.3.3 Damped Sinusoidal Signals

The damped sinusoid combines the exponential envelope with sinusoidal oscillation and arises naturally as the free response of any underdamped second-order linear system:

$$x(t) = Ae^{-\alpha t} \cos(\omega_0 t + \phi), \quad \alpha > 0. \quad (10)$$

The time constant $\tau = 1/\alpha$ governs the envelope decay, while ω_0 governs the oscillation rate. These two parameters are independently controllable in firmware, making this waveform particularly instructive for studying transformation effects.

1.4 Unit Impulse and Unit Step Functions

The unit impulse $\delta(t)$ and unit step $u(t)$ are the two foundational singularity functions in signals and systems theory. While not directly output by the Arduino in this laboratory, their mathematical properties underpin the interpretation of every hardware measurement.

1.4.1 Unit Impulse (Dirac Delta)

$$\delta(t) \triangleq \begin{cases} \infty, & t = 0 \\ 0, & t \neq 0 \end{cases}, \quad \int_{-\infty}^{\infty} \delta(t) dt = 1. \quad (11)$$

The impulse is rigorously defined by its sifting property: for any signal $x(t)$ continuous at $t = t_0$,

$$\int_{-\infty}^{\infty} x(t) \delta(t - t_0) dt = x(t_0). \quad (12)$$

Its discrete-time counterpart, the Kronecker delta, is directly representable in memory:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0. \end{cases} \quad (13)$$

In MATLAB, a Kronecker delta of length N centered at sample k is simply:

```
d = zeros(1, N); d(k+1) = 1; % MATLAB indexing starts at 1
```

1.4.2 Unit Step Function

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}, \quad u(t) = \int_{-\infty}^t \delta(\tau) d\tau. \quad (14)$$

The step function acts as a causal window: multiplying any signal $x(t)$ by $u(t - t_0)$ zeroes it out for $t < t_0$, modeling the physical action of turning on a source at time t_0 . Its discrete counterpart is:

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0. \end{cases} \quad (15)$$

The fundamental calculus relationship between the two singularity functions,

$$\delta(t) = \frac{d}{dt} u(t) \quad \text{and} \quad u(t) = \int_{-\infty}^t \delta(\tau) d\tau, \quad (16)$$

will recur throughout the course when analyzing LTI system impulse and step responses.

2 Pre-Lab Assignment

Provide rigorous mathematical derivations and supporting sketches for the following problems. **Your assigned base signal parameters are listed in Table 2, which your TA will fill in at the start of the laboratory session. Do not proceed with pre-lab work until you have received your seed.**

Table 2. TA-Assigned Base Signal Parameters (Random Seed: _____)

Parameter	Description	Assigned Value
A	Base signal amplitude (V)	_____
f_0	Fundamental frequency (Hz)	_____
ϕ	Initial phase (rad)	_____
α	Exponential decay rate (s^{-1}), if applicable	_____
Signal Type	Sinusoidal / Exponential / Damped Sinusoidal	_____
t_0	Time shift duration (ms)	_____
B	Amplitude scale factor (potentiometer target ratio)	_____

- Pre-lab Deliverable (1/4): Analytical Transformation Derivations:** Using your TA-assigned base signal $x(t)$ (closed-form expression), derive algebraic expressions for the four transformed signals below. Show all intermediate steps, and for each result explicitly state the effect on (a) amplitude, (b) frequency or time constant, (c) initial phase, and (d) causality.
 - Time-shifted signal: $y_1(t) = x(t - t_0)$
 - Amplitude-scaled signal: $y_2(t) = B \cdot x(t)$
 - Time-reversed signal: $y_3(t) = x(-t)$
 - Combined transformation: $y_4(t) = B \cdot x(-t - t_0)$
- Pre-lab Deliverable (2/4): Predicted Waveform Sketches:** On a single set of labeled axes spanning at least $3/f_0$ seconds, sketch all five waveforms $x(t)$, $y_1(t)$, $y_2(t)$, $y_3(t)$, and $y_4(t)$ with accurate amplitudes and periods. Mark the time shift t_0 with a labeled arrow. Use different line styles (solid, dashed, dotted, etc.) to distinguish each signal and include a legend.
- Pre-lab Deliverable (3/4): MATLAB Operator Mapping:** Write the exact MATLAB expressions (using `circshift`, `fliplr`, and scalar operations) that implement transformations y_1 through y_4 on a discrete vector $\mathbf{x}$ of length $N = 512$ sampled at $T_s = 1/(256 f_0)$. Compute the integer shift depth $k = \lfloor t_0/T_s \rfloor$ numerically for your assigned t_0 and f_0 , and state its value explicitly.
- Pre-lab Deliverable (4/4): Unit Step Windowing Analysis:** Express the signal $x_w(t) = x(t) \cdot u(t - t_0)$ using your assigned base signal $x(t)$. Sketch $x_w(t)$, label the onset time t_0 , and in one paragraph explain the physical significance of this operation in the context of causal signal processing.

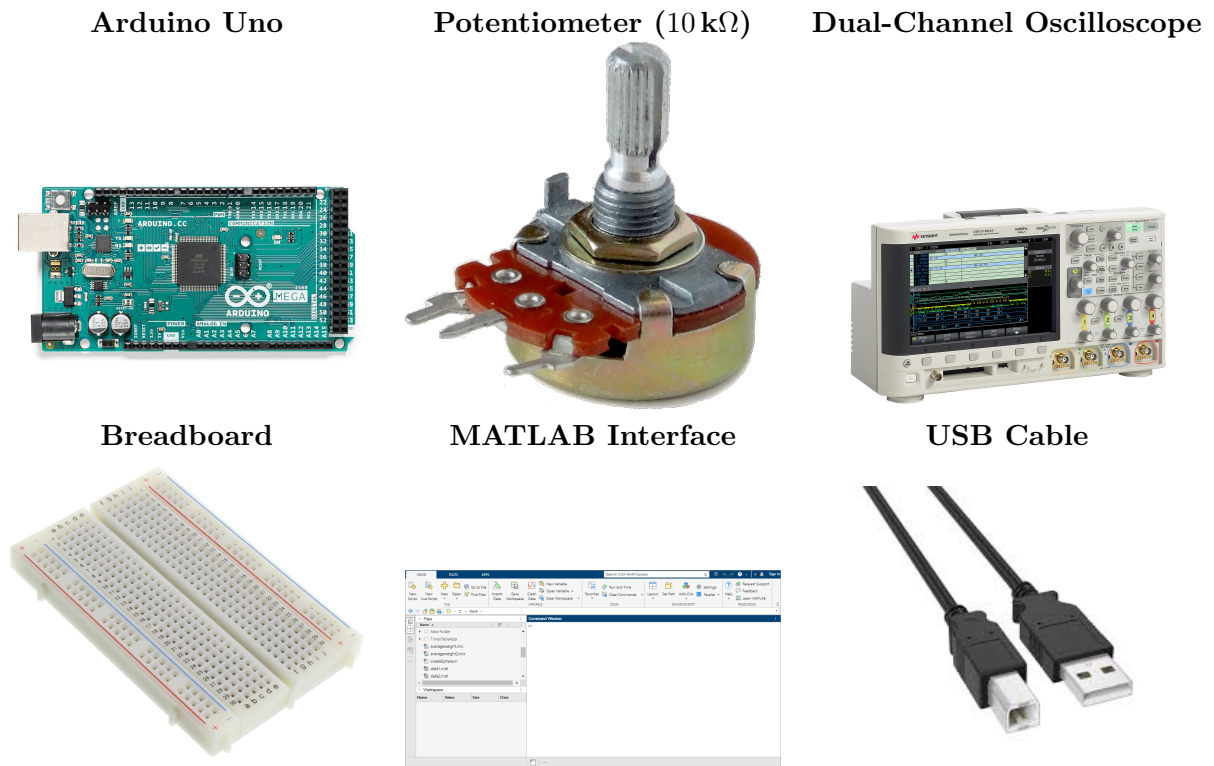
3 Equipment & Setup

Table 3 lists the hardware and software required for this laboratory session.

Table 3. Hardware and DAQ Software Requirements

Classification	Required Tooling/Resources
Compute System	Arduino Microcontroller (Uno or equivalent), Standard USB A-to-B Serial bus.
Analog Control	Prototyping Breadboard, 10 k Ω Linear-Taper Potentiometer, Jumper wires.
Bench Instrumentation	High-resolution Dual-Channel Digital Oscilloscope or ADALM2000 hardware suite.
IDE & Processing	Arduino Studio IDE (C++ deployment), MATLAB (Serial Acquisition and Signal Processing Toolboxes).

Table 4. Visual Reference of Key Equipment



3.1 Potentiometer Wiring and Gain Calculation

The 10 k Ω potentiometer is configured as a variable voltage divider between the Arduino's 5 V supply rail and ground. The wiper (center pin) connects to Arduino analog input A1, providing a continuously variable control voltage $V_{\text{pot}} \in [0, 5]$ V. The normalized gain read by the firmware is:

$$B_{\text{hw}} = \frac{\text{analogRead}(A1)}{1023} \in [0, 1]. \quad (17)$$

Potentiometer Wiring Diagram Placeholder

Pin 1 (left terminal) → Arduino 5V
Pin 2 (wiper / center) → Arduino A1
Pin 3 (right terminal) → Arduino GND

RC filter output (from Lab 1) → Arduino A0 (base signal acquisition)

Figure 1. Potentiometer configured as a gain controller. The wiper voltage is read by `analogRead(A1)` and used to multiply the buffered signal amplitude in real time.

4 Laboratory Procedure

4.1 Submission Evidence Map

Use this map while working to keep your report organized and complete.

Table 5. Required Student Evidence by Procedure Step

Step	Evidence Type	What must be visible
1.1	Oscilloscope photo	Base waveform, time/div, volts/div, measured frequency/amplitude, handwritten name/ID.
1.2	MATLAB screenshot	512-sample base signal plot with labeled axes and title.
2.1	Dual-channel scope photo	Scaled vs original signals with annotated measured gain ratio.
2.2	Dual-channel scope photo	Original vs delayed signal with cursor-measured delay annotation.
2.3	Dual-channel scope photo	Original vs reversed signal, annotated distinguishing feature.
2.4	Dual-channel scope photo	Original vs combined transform with amplitude ratio and shift annotations.
3.1	MATLAB screenshots	Five acquired signals: x , y_1 , y_2 , y_3 , y_4 .
3.2	MATLAB screenshots	Four overlay plots (base, measured, predicted).
3.3	MATLAB output/table	NRMSE values for all four transformations.

4.2 Phase 1: Base Signal Generation and Dual-Channel Acquisition

4.2.1 Step 1.1: Load the TA-Assigned Base Signal

Checkpoint (2/8): Base Signal Verification: Program your Arduino with the TA-provided seed parameters to generate your unique base signal. Verify the output on Channel 1 of the oscilloscope before proceeding to any transformation steps.

Procedure:

1. In MATLAB, run the LUT pre-computation script (Figure 2) using your TA-assigned parameters. Copy the printed LUT array into the Arduino sketch template.
2. Flash the Arduino. Connect the RC-filtered PWM output to both oscilloscope Channel 1 and Arduino A0.
3. Verify the oscilloscope displays the correct waveform morphology (sinusoidal, exponential, or damped sinusoidal), with frequency $f_0 \pm 5\%$ and amplitude $A \pm 5\%$.
4. Record the measured frequency and peak-to-peak amplitude in your lab notebook.

Deliverable (3/12): Submit an oscilloscope screenshot of the base signal. The image must show: (a) measured frequency readout, (b) measured peak-to-peak amplitude, (c) time/div and volts/div settings, and (d) your **handwritten name and student ID** visible in the frame. This is your primary authenticity proof.

STUDENT EVIDENCE PLACEHOLDER (Step 1.1): Base-signal oscilloscope screenshot with name/ID visible

REFERENCE — MATLAB LUT Pre-Computation from TA Seed:

% Run in MATLAB before the lab session. Paste the printed LUT into Arduino.

```
rng(ASSIGNED_SEED);           % <-- replace with your TA-assigned integer seed
N   = 256;
f0   = ASSIGNED_F0;           % Hz
A    = ASSIGNED_A;            % V
phi  = ASSIGNED_PHI;          % rad
alph = ASSIGNED_ALPHA;        % s-1 (set 0 for pure sinusoidal)

t     = linspace(0, 1/f0, N+1);
t     = t(1:N);               % one period, N samples, exclude endpoint

% Select signal type based on TA assignment:
% Sinusoidal:      x = A * cos(2*pi*f0*t + phi);
% Exponential:    x = A * exp(alph * t);    (alph < 0 for decay)
% Damped sinusoid: x = A * exp(-alph*t) .* cos(2*pi*f0*t + phi);

x      = A * exp(-alph * t) .* cos(2*pi*f0*t + phi); % edit as needed
x_norm = (x - min(x)) ./ (max(x) - min(x));          % normalize to [0, 1]
lut     = uint8(round(x_norm * 255));                 % 8-bit DAC range

% Print array for copy-paste into Arduino sketch
fprintf('const byte lut[256] PROGMEM = {');
fprintf('%d, ', lut(1:end-1));
fprintf('%d};\n', lut(end));
```

Figure 2. REFERENCE MATLAB LUT generator. Replace all `ASSIGNED_*` placeholders with your TA-assigned values. The printed array is pasted verbatim into the Arduino sketch.

4.2.2 Step 1.2: Acquire the Base Signal into MATLAB

Checkpoint (3/8): MATLAB Base Signal Acquisition: Acquire 512 contiguous samples of the base signal via UART serial at baud rate 115200 and produce a calibrated voltage-versus-time plot.

Procedure: Configure the Arduino to transmit `analogRead(A0)` values at your assigned sampling rate $f_s = 256 f_0$. Run the MATLAB acquisition script in Figure 3 to collect the base signal vector `x_hw`.

Deliverable (4/12): Submit a MATLAB figure of the acquired base signal `x_hw` with: correct time axis in milliseconds, voltage axis in volts, grid enabled, and a descriptive title. Confirm visually that the MATLAB waveform matches the oscilloscope capture from Step 1.1.

STUDENT EVIDENCE PLACEHOLDER (Step 1.2): MATLAB base-signal plot screenshot

REFERENCE — MATLAB Serial Acquisition Script:

```
clear; clc;
%--- Serial setup ---
port = 'COM3';          % change to match your system
baud = 115200;
f0    = ASSIGNED_F0;    % Hz -- your TA-assigned frequency

s = serialport(port, baud);
configureTerminator(s, "CR/LF");

%--- Acquire 512 samples ---
N    = 512;
raw = zeros(N, 1);
for k = 1:N
    raw(k) = str2double(readline(s));
end
clear s;

%--- Convert ADC counts to voltage ---
x_hw = (raw / 1023) * 5.0;    % volts
Ts    = 1 / (256 * f0);      % sampling period (s)
t     = (0:N-1) * Ts;        % time axis (s)

%--- Plot ---
figure('Position', [100 100 850 400]);
plot(t*1e3, x_hw, 'b-', 'LineWidth', 1.5);
xlabel('Time (ms)', 'FontSize', 12);
ylabel('Voltage (V)', 'FontSize', 12);
title('Base Signal x_{hw}[n] -- Acquired from Arduino', 'FontSize', 14);
grid on;
ylim([0 5.5]);
```

Figure 3. REFERENCE acquisition script. Modify `port` and `ASSIGNED_F0` to match your setup.

Discussion (1/9): Compare the oscilloscope capture (Step 1.1) with the MATLAB plot (Step 1.2). Identify and quantify discrepancies in frequency, amplitude, and waveform morphology. Compute the amplitude percent error $\varepsilon_A = |A_{\text{scope}} - A_{\text{MATLAB}}| / A_{\text{scope}} \times 100\%$ and propose the dominant physical cause.

4.3 Phase 2: Hardware Implementation of Signal Transformations

4.3.1 Step 2.1: Amplitude Scaling via Potentiometer

Checkpoint (4/8): Potentiometer Gain Stage: Wire the 10k Ω potentiometer as described in Section 5 and implement firmware that multiplies the LUT output by the normalized potentiometer reading in real time.

Procedure:

1. Wire the potentiometer to A1 per the schematic in Figure 1.
2. Modify the Arduino ISR to apply the gain: `out_val = (pot / 1023.0) * lut_val` (see Figure 4 for the complete reference ISR).
3. Adjust the potentiometer until the oscilloscope measures $B \cdot A$ on Channel 1. Simultaneously display the unscaled reference (original signal) on Channel 2.
4. Confirm the amplitude ratio matches your TA-assigned B to within 10%.

Deliverable (5/12): Submit a dual-channel oscilloscope screenshot with Channel 1 showing the scaled signal $y_2(t) = B \cdot x(t)$ and Channel 2 showing the original $x(t)$. Annotate the measured amplitudes of both channels and the computed ratio $B_{\text{meas}} = A_{\text{Ch1}}/A_{\text{Ch2}}$.

STUDENT EVIDENCE PLACEHOLDER (Step 2.1): Scope capture for amplitude scaling

```

REFERENCE — Arduino Amplitude-Scaling ISR:

#define PWM_PIN 3 // original signal output

volatile byte lut_idx = 0;
volatile float gain = 0.5f; // updated in loop() from A1 reading
const byte lut[256] PROGMEM = { /* paste your TA LUT here */ };

// ISR: called by Timer1 at rate f_s = 256 * f0
ISR(TIMER1_COMPA_vect) {
    byte lut_val = pgm_read_byte(&lut[lut_idx]);
    byte out_val = (byte)(gain * lut_val); // apply gain
    analogWrite(PWM_PIN, out_val);
    lut_idx = (lut_idx + 1) & 0xFF; // increment and wrap (0..255)
}

void loop() {
    // Read potentiometer outside ISR to avoid blocking interrupt timing
    gain = analogRead(A1) / 1023.0f;
    // Note: 'gain' must be declared volatile for ISR-safe sharing
}

```

Figure 4. REFERENCE ISR for potentiometer-controlled amplitude scaling. The `gain` variable is updated asynchronously in `loop()` to avoid extending ISR execution time.

Discussion (2/9): The reference code reads the potentiometer in `loop()` rather than inside the ISR. Explain why placing `analogRead(A1)` inside the ISR would introduce a timing hazard. What C++ storage qualifier must the shared variable `gain` carry, and what does that qualifier instruct the compiler to do differently? How does this prevent a subtle race condition between the main code and the ISR?

4.3.2 Step 2.2: Time Shift via Circular Sample Buffer

Checkpoint (5/8): Time-Shift Buffer: Implement a k -sample circular FIFO delay buffer in firmware to produce the time-shifted signal $y_1[n] = x[n - k]$.

Procedure:

1. Compute the buffer depth: $k = \lceil t_0/T_s \rceil$, where $T_s = 1/(256 f_0)$ seconds. Verify this matches your pre-lab value.
2. Implement the circular FIFO buffer (Figure 5). Set `K_DELAY` to your computed k .
3. Output the original signal on `PWM_PIN` and the delayed signal on `PWM_PIN_DELAYED`. Capture both simultaneously on the dual-channel oscilloscope.
4. Use the oscilloscope's cursor or automatic measurement tool to quantify the time between corresponding features (e.g., zero-crossings or peaks) on the two channels.

Deliverable (6/12): Submit a dual-channel oscilloscope screenshot with Channel 1 showing $x(t)$ and Channel 2 showing $y_1(t) = x(t - t_0)$. Using cursors, measure and annotate the delay between corresponding zero-crossings or peaks. Compare to your pre-lab predicted t_0 .

STUDENT EVIDENCE PLACEHOLDER (Step 2.2): Scope capture for time shift with cursor delay annotation

REFERENCE — Circular Delay Buffer (Arduino):

```
#define PWM_PIN          3          // original
#define PWM_PIN_DELAYED  5          // delayed -- use second PWM-capable pin
#define K_DELAY          26         // buffer depth = floor(t0 / Ts); set your value

volatile byte circ_buf[K_DELAY];
volatile byte buf_head = 0;

ISR(TIMER1_COMPA_vect) {
    byte lut_val = pgm_read_byte(&lut[lut_idx]);

    // FIFO: read the oldest sample (delayed output), then overwrite it
    byte delayed_val = circ_buf[buf_head];
    circ_buf[buf_head] = lut_val;
    buf_head = (buf_head + 1) % K_DELAY;

    analogWrite(PWM_PIN, lut_val);      // original x[n]
    analogWrite(PWM_PIN_DELAYED, delayed_val); // shifted x[n-k]

    lut_idx = (lut_idx + 1) & 0xFF;
}
```

Figure 5. REFERENCE circular FIFO delay buffer. Replace `K_DELAY` with your computed shift depth k .

Discussion (3/9): For a sinusoidal base signal $x(t) = A \cos(2\pi f_0 t)$, show algebraically that a time delay $t_0 = 1/(4f_0)$ is exactly equivalent to a phase shift of $-\pi/2$ radians (quarter-period lag). Does

this equivalence hold for a non-sinusoidal signal such as the decaying exponential $x(t) = Ae^{-\alpha t}$? Justify your answer rigorously in terms of the definition of “phase.” Why is phase a concept that applies to sinusoidal signals specifically?

4.3.3 Step 2.3: Time Reversal via Reversed LUT Indexing

Checkpoint (6/8): Time Reversal Implementation: Implement time reversal by decrementing the LUT index rather than incrementing it, effectively reading the stored waveform backwards in time.

Procedure:

1. Modify the ISR to traverse the LUT in reverse: decrement `rev_idx` each tick, wrapping at 0 back to 255 (see Figure 6).
2. Output the forward signal on `PWM_PIN` and the reversed signal on `PWM_PIN_REV`.
3. Capture both channels on the oscilloscope.
4. Describe qualitatively how the reversed waveform differs from the original. For a symmetric base signal (e.g., a zero-phase cosine), explain what you expect to see.

Deliverable (7/12): Submit a dual-channel oscilloscope screenshot with Channel 1 showing $x(t)$ and Channel 2 showing $y_3(t) = x(-t)$. For an asymmetric waveform (e.g., a damped sinusoidal), annotate the key visual feature that distinguishes the reversed signal from the original.

STUDENT EVIDENCE PLACEHOLDER (Step 2.3): Scope capture for time reversal

```

REFERENCE — Reversed LUT Indexing ISR:

#define PWM_PIN      3
#define PWM_PIN_REV  5

volatile byte fwd_idx = 0;
volatile byte rev_idx = 255;    // start at last sample of LUT

ISR(TIMER1_COMPA_vect) {
    byte orig_val = pgm_read_byte(&lut[fwd_idx]);    // forward: x[n]
    byte rev_val  = pgm_read_byte(&lut[rev_idx]);    // reversed: x[-n]

    analogWrite(PWM_PIN,      orig_val);
    analogWrite(PWM_PIN_REV, rev_val);

    fwd_idx = (fwd_idx + 1) & 0xFF;    // increment with wrap
    rev_idx = (rev_idx - 1 + 256) & 0xFF;    // decrement with wrap
}

```

Figure 6. REFERENCE reversed-index ISR. Both the forward and time-reversed signals are output simultaneously, enabling direct dual-channel oscilloscope comparison.

Discussion (4/9): Under what conditions on $x(t)$ does $x(-t) = x(t)$? This property defines an even function. Under what conditions does $x(-t) = -x(t)$? This defines an odd function. Determine analytically whether your assigned base signal is even, odd, or neither, and explain what the dual-channel oscilloscope display should look like in each case.

4.3.4 Step 2.4: Combined Transformation — Reversal, Shift, and Scale

Checkpoint (7/8): Combined Transformation Verification: Implement $y_4(t) = B \cdot x(-t - t_0)$ in firmware by composing the three individual transformations from Steps 2.1–2.3. Verify the result against your pre-lab prediction.

Procedure:

1. Integrate the reversed LUT index (Step 2.3), the circular delay buffer (Step 2.2), and the potentiometer gain (Step 2.1) into a single ISR.
2. Output $x(t)$ on Channel 1 and $y_4(t)$ on Channel 2 of the oscilloscope.
3. Set the potentiometer to the TA-assigned B value.
4. Compare the oscilloscope output qualitatively and quantitatively to your pre-lab sketch of $y_4(t)$.

Deliverable (8/12): Submit a dual-channel oscilloscope screenshot of $x(t)$ (Ch. 1) versus $y_4(t) = B \cdot x(-t - t_0)$ (Ch. 2). Annotate: (a) measured amplitude ratio B_{meas} , (b) measured time offset between corresponding features, and (c) whether the reversed morphology matches your pre-lab sketch.

**STUDENT EVIDENCE PLACEHOLDER (Step 2.4): Scope capture for
combined transformation**

4.4 Phase 3: MATLAB Verification and Predicted vs. Measured Overlay

4.4.1 Step 3.1: Acquire All Transformed Signals into MATLAB

Checkpoint (8/8): Full Transformation Acquisition: Acquire 512-sample MATLAB vectors for each of the four transformed signals $y_1[n]$ through $y_4[n]$. Each signal should be acquired with the Arduino running the corresponding transformation from Phase 2.

Deliverable (9/12): Submit MATLAB figures for all five signals (x , y_1 , y_2 , y_3 , y_4), each with: time axis in milliseconds, voltage axis in volts, grid enabled, and a descriptive title identifying the transformation and your assigned parameters.

STUDENT EVIDENCE PLACEHOLDER (Step 3.1): MATLAB screenshots
for all acquired transformed signals

4.4.2 Step 3.2: Predicted vs. Measured Overlay Plots

Deliverable (10/12): For each of the four transformations, produce a single MATLAB figure that overlays three signals: (1) the base signal x_{hw} (blue solid), (2) the hardware-acquired transformation y_{meas} (red dashed), and (3) the MATLAB-predicted transformation y_{pred} computed from Table 1 (black dotted). Include a legend identifying all three traces. See Figure 7 for the reference overlay template.

STUDENT EVIDENCE PLACEHOLDER (Step 3.2): MATLAB overlay
screenshots for y_1 through y_4

REFERENCE — MATLAB Overlay: Measured Hardware vs. Predicted Theory:

```

% Assumes x_hw, Ts, t, and assigned parameters are already in workspace
k      = floor(t0_s / Ts);           % shift depth (integer samples)
B_val  = ASSIGNED_B;                % potentiometer target gain

% Predicted transformations from base vector
y1_pred = circshift(x_hw, k);        % time shift
y2_pred = B_val * x_hw;              % amplitude scale
y3_pred = fliplr(x_hw);              % time reversal
y4_pred = B_val * circshift(fliplr(x_hw), k); % combined

% -- Example: Time Shift Overlay --
figure('Position', [100 100 900 430]);
plot(t*1e3, x_hw, 'b-', 'LineWidth', 1.5, 'DisplayName', 'x_{hw} (base)');
hold on;
plot(t*1e3, y1_meas, 'r--', 'LineWidth', 1.5, 'DisplayName', 'y_1 (hardware)');
plot(t*1e3, y1_pred, 'k:', 'LineWidth', 2.0, ...
     'DisplayName', sprintf('y_1 = circshift(x,%d) (predicted)', k));
xlabel('Time (ms)', 'FontSize', 12);
ylabel('Voltage (V)', 'FontSize', 12);
title(sprintf('Time Shift Verification: t_0 = %.1f ms', t0_s*1e3), ...
      'FontSize', 14);
legend('Location', 'best'); grid on;

% Repeat the same overlay structure for y2_pred, y3_pred, y4_pred

```

Figure 7. REFERENCE overlay template for the time-shift verification. Replicate this structure for y_2 , y_3 , and y_4 .

4.4.3 Step 3.3: Residual Error Quantification

Deliverable (11/12): For each of the four transformations, compute the Normalized Root Mean Square Error (NRMSE) between the hardware-acquired signal and the MATLAB-predicted signal:

$$\text{NRMSE}(\%) = \frac{\sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (y_{\text{hw}}[n] - y_{\text{pred}}[n])^2}}{\max(y_{\text{pred}}) - \min(y_{\text{pred}})} \times 100\%. \quad (18)$$

```

% MATLAB NRMSE calculation
nrmse = @(hw, pred) rms(hw(:) - pred(:)) / (max(pred) - min(pred)) * 100;
fprintf('Time Shift      NRMSE: %.2f%%\n', nrmse(y1_meas, y1_pred));
fprintf('Ampl. Scale    NRMSE: %.2f%%\n', nrmse(y2_meas, y2_pred));
fprintf('Time Reversal  NRMSE: %.2f%%\n', nrmse(y3_meas, y3_pred));
fprintf('Combined       NRMSE: %.2f%%\n', nrmse(y4_meas, y4_pred));

```

**STUDENT EVIDENCE PLACEHOLDER (Step 3.3): NRMSE output table
or MATLAB command-window screenshot**

Discussion (5/9): Which of the four transformations yields the largest NRMSE? Propose and justify at least two distinct physical or algorithmic mechanisms that cause the hardware output to deviate from the MATLAB prediction. Distinguish between errors that are systematic (biased, repeatable) versus random (stochastic, non-repeatable) and state how you would characterize each from the available data.

5 Data Analysis

5.1 Analysis Step 1: Pre-Lab Sketch vs. Hardware vs. MATLAB Comparison

Discussion (6/9): For each of the four transformations, compare your pre-lab hand-sketch against the oscilloscope capture and the MATLAB-measured signal. Create a completed version of Table 6 and identify which domain shows the largest deviation from the theoretical prediction. Explain the primary source of discrepancy in each case.

Table 6. Transformation Summary Table (complete in your report)

Parameter	Theoretical (Pre-Lab)	Hardware Measured	% Error
Base amplitude A (V)			
Base frequency f_0 (Hz)			
Scaled amplitude $B \cdot A$ (V)			
Measured gain ratio B_{meas}			
Time shift t_0 (ms)			
Phase shift $\phi_0 = \omega_0 t_0$ (rad)			
Combined amplitude $B \cdot A$ (V)			
Combined shift t_0 (ms)			

5.2 Analysis Step 2: Signal Symmetry Classification

Discussion (7/9): Classify your assigned base signal $x(t)$ as even, odd, or neither. Support the classification analytically by evaluating whether $x(-t) \stackrel{?}{=} x(t)$ or $x(-t) \stackrel{?}{=} -x(t)$. Confirm visually by overlaying the MATLAB vectors `x_hw` and `fliplr(x_hw)` on a single plot. Any even signal should show perfect overlap; any odd signal should show perfect sign inversion.

5.3 Analysis Step 3: circshift Boundary Artifact Investigation

Discussion (8/9): `circshift` performs a circular shift: samples that scroll off one end of the array reappear at the other. (a) Identify in your overlay plot where this wrap-around artifact is visible. (b) Explain the mathematical reason it occurs. (c) Propose a practical mitigation: describe how zero-padding the vector to length $2N$ before calling `circshift` would eliminate the artifact and demonstrate this in MATLAB with a brief code snippet.

5.4 Analysis Step 4: Unit Step Windowing Verification

Discussion (9/9): Using your acquired base signal vector `x_hw`, construct the MATLAB representation of $x_w[n] = x[n] \cdot u[n - k]$ by zeroing all samples with index $n < k$. Plot both `x_hw` and `x_w` on the same axes. In one paragraph, explain the connection between the unit step function, causal signal processing, and the concept of switching a signal source on at a specified time t_0 .

5.5 Analysis Step 5: Error Source Ranking

Deliverable (12/12): Using the NRMSE values from Step 3.3 and the measurements in Table 6, rank the three largest sources of error (by estimated contribution to NRMSE) from the following list: PWM quantization noise, DAC non-linearity, RC filter phase distortion, ADC quantization, UART serial transmission jitter, potentiometer contact noise, and Timer1 interrupt jitter. For each

identified source, provide a one-sentence physical justification for its ranking, referencing specific numerical evidence from your measurements.

6 Grading Rubric

Total Points: 100

Table 7. Detailed Grading Rubric with Specific Criteria

Category	Evaluation Criteria	Weight
Pre-Lab (20 pts)	PL1: Closed-form derivations for y_1 - y_4 with effects on amplitude/frequency/phase/causality	8
	PL2: Five waveform sketches with correct axes, labels, legend, and t_0 arrow	6
	PL3: MATLAB operator expressions with correct shift depth k computed numerically	4
	PL4: Unit step windowing derivation, sketch, and causal-processing explanation	2
Base Signal (10 pts)	Scope photo: name/ID visible, frequency $\leq 5\%$ error, amplitude $\leq 5\%$ error	5
	MATLAB acquisition plot: correct axes (ms, V), grid, descriptive title	5
Hardware Transformations (28 pts)	Amplitude scaling: dual-channel scope, B_{meas} annotated ($\leq 10\%$ error)	7
	Time shift: dual-channel scope, delay measured by cursor ($\leq 10\%$ error)	7
	Time reversal: dual-channel scope, asymmetry feature annotated	7
	Combined y_4 : scope capture, amplitude and shift both annotated vs. pre-lab	7
MATLAB Verification (22 pts)	Four overlay plots (base + hardware + predicted) with legend, axes, grid	12
	NRMSE table: all four values computed, reported, and interpreted	10
Analysis (20 pts)	Table 6 complete with % errors and interpretation of dominant deviations	8
	Symmetry analysis + <code>circshift</code> boundary artifact discussion with mitigation code	6
	Error source ranking: three sources ranked by NRMSE evidence with physical justifications	6
Totals		100

References and Inspiration

This laboratory is part of Duke ECE 280's comprehensive signal processing redesign, drawing on best practices in signal transformation pedagogy from leading universities.

University Course Inspirations

MIT 6.003: Signals, Systems, and Transforms

Lab 02 is inspired by MIT's rigorous treatment of signal transformations (time-shift, amplitude scaling, time-reversal) and their effect on signal properties and spectra. MIT's emphasis on predicting transformation outcomes mathematically before observing them experimentally informs the pre-lab calculations and verification methodology.

Georgia Tech ECE 2026: Real-Time DSP Implementation

Georgia Tech's philosophy of implementing signal transformations in real hardware—not just simulation—shapes this lab's hardware-focused approach. Using the Arduino PWM to apply amplitude scaling and frequency shifting, then validating with oscilloscope measurements, mirrors Georgia Tech's commitment to making abstract signal operations concrete and observable.

Rice University ELEC 241: Real-Time Signal Processing on Embedded Systems

The emphasis on time-precision (microsecond-scale timing for frequency shifting) and the use of embedded timers reflects Rice's hands-on exploration of how signal transformations are implemented under real-time constraints on resource-limited platforms.

Duke University ECE 280: Lab Modernization

This lab is part of Duke's effort to modernize the signals and systems curriculum with hardware implementations that validate theoretical transformations in real time, making signal processing principles tangible to students.

References

- A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter 2: Time and Frequency Domain Representations.
- A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Pearson, 1997. Chapter 1: Signals and Systems (transformations and superposition).
- J. H. McClellan, R. W. Schaffer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015.
- Arduino PWM Reference: <https://www.arduino.cc/en/Reference/AnalogWrite>